

Numpy

Numpy is a low level python library written in C for high level Mathematical functions. Numpy cleverly overcomes the problem of running slower algorithms on python by using multidimensional arrays and functions that operate on arrays.

Applications of Numpy

- A powerful N-dimensional array object
- sophisticated function
- tools for integrating C/C++
- useful linear algebra, Fourier transform and random number capabilities.

N-d array properties:

- Homogeneous
- Only numbers
- Fixed item size.

→ Importing Numpy library

import numpy as np. → Adding numpy lib features to our python code.
└─> nick name for library we kept.

`np.array([1,2,3,4,5])`

→ using this command we can send list and that will create array for us.

• `np.zeros((2,3))`

→ using this we can make N-D array using tuple as input but it have all zeros in it

• `np.ones((2,3))`

↳ works same as `np.zeros` but initializes all values as one.

• `np.identity(5)`

↳ single number is inserted. creates an identity matrix.

• `np.arange(10)`

↳ creates a array from the range we give. we can also decide the gaps between them.

• `np.linspace(10,20,10)`

↳ we tell that how much equidistant points to make between them.

• `np.copy(a)`

→ copy an N-D array.

Numpy array properties and attributes.

`arr.shape` → tells the dimension edges of the array.

`arr.ndim` - tells the dimension of that array.

`arr.size` - tells us the no. of items in that array.

`arr.itemsize` → tells how much memory the single item inside arr is taking.

`arr.dtype` → tells the datatype of the items in the array.

`arr.astype()` → to change the datatype of the items inside.

(Numpy arrays vs Python list)

Array are faster, convenient, Less Memory.

Indexing, slicing & iteration

Indexing

- `reshape` → we can use it to change the dimension of our arrays and select its edges.

`arr[1]`



gives element
on that index

`arr[1:3]`



gives the
range of element

`arr[1, 2]`



gives the
element inside
the element.

`arr[:, 2]`

↳ to extract a particular column.

Iteration

`np.nditer(arr12)`: used in for loop to iterate all the element of the nd array.

→ Numpy operations

we can perform operation on single array

`arr * 2` → Multiplied every element.

we can perform on arrays also

`arr1 * arr2` → element wise multiplication.

→ `arr1.dot(arr2)`

↳ gives the dot product (Matrix
Multiplication)
of the arrays

→ `arr.max()`

gives maximum element of that array

→ `arr.min()` → gives minimum.

→ we can use axis to get column & row functions. (axis = 0 → column)
(axis = 1 → row)

→ `arr.sum()` → adds all the

↳ we can use axis also.

→ `arr.mean()` → to calculate mean of the

→ `arr.std()` → gives standard deviation.

→ `np.sin(arr)` → to find sin of all values

→ `np.median(arr)` → to find median.

Reshaping numpy array

→ Ravel → converts any array to 1-D array

`arr.ravel()`

→ Reshape

↳ we can use to change the dimension of the arr.

`arr.reshape(2,5)`

↳ arr must have 10 element

→ transpose → Give transpose of a N-D array

`arr4.transpose()`

→ stacking

we can combine 2 arrays but they must be of same shape.

→ `np.hstack((arr1, arr2))`

→ `np.vstack((arr1, arr2))`

vertically
stacks

horizontally
stacks.

→ Splitting

`np.vsplit(arr, n)`

`np.hsplit(arr, n)`

no. of equal parts.

Indexing with boolean array

`random.randint` → used to generate N-D array of random number.

`arr[arr > 50]`

↳ this can be used to find particular elements from the array.

Broadcasting

Ability of Numpy to treat array of different shapes during arithmetic operations.

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 5 & 5 & 5 \end{bmatrix} = \begin{bmatrix} 6 & 7 & 8 \end{bmatrix}$$

↳ virtually we can see how it makes extra duplicated elements and match them.

If 1 is there at start of the shape and the y axis of them are of some dimension then it will take place.

If shape of one array is $(1, n)$ so second one must be (y, n) or $(n, 1)$ to make broadcasting happen.

If both are $(1, 1)$ then it can be broadcasted with any array.

→ Important functions of Numpy.

→ `np.random.random()`

→ gives any random value between $(0-1)$

→ `np.random.seed()`

↳ this gives us the same random variable again and again.

→ `np.random.uniform(1, 10)`

↳ gives a random variable in the given range.
we can also tell how many elements we want.

→ `np.argmax()`

↳ gives index of maximum value.

→ `np.where(a%2==0, -1, 1)` ^{if % false}

↳ used to return a new array and don't change original array.
↳ if true

→ `np.sort(arr)` → can be used to sort the array

→ `n.percentile(a, 50)`

↳ can be used to give number

at that particular percentile.

x x x x Thanks. x x x x x